Computational Neuroscience — Week 6

Computing with Networks

Modelling Synapses, and Wiring Neurons Together

These notes cover lectures 1–3 — putting a synapse into the membrane equation, wiring two neurons together, abstracting from spikes to firing rates so that whole networks can be modelled, and analysing what recurrent networks do. Green boxes give the plain-words version of each idea before the mathematics; rust boxes flag the places where the obvious reading is the wrong one.

Contents

I Modelling Synapses **2**

1 Where We Are **2**

2 What a Synapse Does **2**

3 The Membrane, Recapped **3**

4 Putting a Synapse In **4**

5 What Sets the Synaptic Conductance **4**

6 Many Spikes: the Linear Filter Again **5**

7 A Two-Neuron Network **6**

8 Where This Leaves Us **7**

II From Spikes to Firing Rates **7**

9 Spiking or Firing Rate? **7**

10 Bridging the Two **7**

11 The Firing-Rate Model **8**

12 Many Outputs, and Feedback **9**

13 What a Linear Feedforward Network Computes **9**

14 The Brain Does Calculus **10**

15 Where This Leaves Us **10**

III Recurrent Networks **11**

16 The Question **11**

17 Eigenvectors to the Rescue **11**

18 Eigenvalues Determine Stability 12

19 Amplification 12

20 A Concrete Example: the Cosine Network 13

21 Memory: What Happens at $\lambda = 1$ 13

22 Nonlinear Recurrent Networks 14

23 Non-Symmetric Networks 15

24 Linear Stability Analysis 15

25 Where This Leaves Us 16

Part I

Modelling Synapses

Lecture 1 — synaptic conductance, the kinetic model, linear filtering, and a two-neuron network.

1. Where We Are

[Lecture slide omitted from the public copy.] Five weeks in one slide.

The idea in plain words

The course has run in two halves. Weeks 2–4 built *descriptive* models — what does a neuron encode, how do you decode it, how much information does it carry. Week 5 switched to *mechanistic* models of a single cell: the RC circuit, Hodgkin–Huxley, integrate-and-fire. Everything so far has been one neuron. This week connects them.

2. What a Synapse Does

[Lecture slide omitted from the public copy.] Neurons connect through synapses. We focus on ***chemical*** *synapses, which are much the more common kind.*

A spike in the **presynaptic** neuron causes neurotransmitter to be released into the **synaptic cleft**. It binds to receptors on the **postsynaptic** membrane, which opens ion channels, which changes the postsynaptic membrane potential — up or down depending on the synapse.

[Lecture slide omitted from the public copy.] The net effect: increase or decrease the postsynaptic membrane potential.

[Lecture slide omitted from the public copy.] An ***excitatory*** *synapse. Glutamate opens channels that let positive ions such as Na^+ in, which* ***depolarises*** *the cell.*

*[Lecture slide omitted from the public copy.] An **inhibitory** synapse. GABA opens channels that let Cl^- in or K^+ out, which **hyperpolarises** the cell.*

The idea in plain words

Both synapse types do exactly the same *thing* — release a transmitter that opens channels. What differs is *which* channels, and therefore which way the resulting current pushes the voltage. That distinction will turn out to be a single parameter in the model.

3. The Membrane, Recapped

Before adding a synapse we need the equation it plugs into.

[Lecture slide omitted from the public copy.] The RC circuit model from Week 5, rederived. Charge on a capacitor is $Q = C_m V$; differentiating gives the current, $dQ/dt = C_m\, dV/dt$.

Writing the current as the injected term plus the leak:

$$c_m \frac{dV}{dt} = -\frac{(V - E_L)}{r_m} + \frac{I_e}{A}$$

and multiplying through by the specific membrane resistance r_m:

$$\tau_m \frac{dV}{dt} = -(V - E_L) + I_e R_m$$
$$\tau_m = r_m c_m = R_m C_m$$

Watch out — lower case and upper case are not the same quantities

r_m and c_m are *specific* — per unit area. R_m and C_m are totals for the whole cell, related by the surface area A: $R_m = r_m/A$ and $C_m = c_m A$. The area cancels in the product, which is why τ_m comes out the same either way.

What the equation says

[Lecture slide omitted from the public copy.] Injecting a constant current drives the voltage to a plateau; switching it off lets it decay back exponentially.

Set $dV/dt = 0$ to find where it settles:

$$0 = -(V - E_L) + I_e R_m \qquad \Longrightarrow \qquad V_{ss} = E_L + I_e R_m$$

The idea in plain words

This is the Week 5 result with the resting potential put back in. There the membrane charged from zero toward IR; here it starts at $E_L \approx -70\,\text{mV}$ and the injected current shifts it by $I_e R_m$ from there.
And τ_m governs how quickly the cell reacts. A large τ_m means sluggish — slow to reach the plateau and slow to return. The lecture's own analogy: a large time constant is you before your morning coffee, a small one is you after.

4. Putting a Synapse In

[Lecture slide omitted from the public copy.] The question. We have a membrane model and we know synapses open channels — so what goes in the circuit?

[Lecture slide omitted from the public copy.] The hint: Hodgkin–Huxley already solved this shape of problem, by adding a conductance *branch for each ion.*

The idea in plain words

A synapse opens channels. Hodgkin–Huxley modelled opening channels by adding a conductance with its own reversal potential. So do the same thing: give the synapse a **synaptic conductance** g_s and a **reversal potential** E_s, and hang it off the membrane as one more branch.

[Lecture slide omitted from the public copy.] The membrane equation with a synaptic term added.

$$\tau_m \frac{dV}{dt} = -\Big[(V - E_L) + g_s\,(V - E_s)\,r_m\Big] + I_e R_m$$

Excitatory or inhibitory is one parameter

Excitatory: $E_s > E_L$, so the synaptic current drags V *up*.
Inhibitory: $E_s < E_L$, so it drags V *down*.
Typical values used later: $E_s = 0\,\text{mV}$ for excitatory, $-80\,\text{mV}$ for inhibitory, against $E_L = -70\,\text{mV}$.

The idea in plain words

This is the same $g_i(V - E_i)$ structure as every ionic branch in Week 5, and it works the same way: the conductance pulls the membrane toward its own reversal potential. Whether that is excitation or inhibition depends entirely on which side of rest E_s sits — not on the transmitter, not on the ion.

5. What Sets the Synaptic Conductance

g_s is not constant — it responds to incoming spikes. It factors into three pieces:

$$g_s = g_{s,\max}\; P_{\text{rel}}\; P_s$$

- $g_{s,\max}$ — the **maximum conductance**, set by how many channels the postsynaptic side has.
- P_{rel} — the **probability of transmitter release** given an input spike.
- P_s — the **probability that postsynaptic channels open**, which is also the fraction of channels open at any moment.

Watch out — P_{rel} is not 1 in a real synapse

A presynaptic spike does *not* always release transmitter — real synapses fail, sometimes often. The model below sets $P_{\text{rel}} = 1$ for simplicity, which is a real simplification, not a

technicality.

The kinetic model

[Lecture slide omitted from the public copy.] Channels flip between closed and open at rates α_s and β_s.

$$\frac{dP_s}{dt} = \underbrace{\alpha_s\,(1-P_s)}_{\text{opening}\times\text{fraction closed}} - \underbrace{\beta_s\,P_s}_{\text{closing}\times\text{fraction open}}$$

The idea in plain words

This is *exactly* the gating equation from Hodgkin–Huxley, with α_s, β_s in place of α_n, β_n. Same two-state bookkeeping: the open fraction grows in proportion to what is currently closed and shrinks in proportion to what is currently open.
The one difference is what drives the rates. In Hodgkin–Huxley they depended on *voltage*; here they are set by the arrival of *transmitter*.

What P_s actually looks like

[Lecture slide omitted from the public copy.] Measured time courses for three synapse types after a single spike.

- **AMPA** and **GABA**$_\text{A}$ are fitted well by a simple exponential, $K(t) = e^{-t/\tau_s}$ — a jump at the spike, then decay.
- **NMDA** is fitted better by an **alpha function**, which rises to a peak *after* the spike before decaying:

  $$\alpha(t) = \frac{t}{\tau_\text{peak}}\, e^{\,1-t/\tau_\text{peak}}$$

  with τ_peak setting the delay to the peak.

The idea in plain words

Note the timescales on the two panels: AMPA and GABA$_\text{A}$ decay within tens of milliseconds, NMDA over hundreds. Same framework, conductances lasting an order of magnitude apart — which is part of why different receptor types do different computational jobs.

6. Many Spikes: the Linear Filter Again

[Lecture slide omitted from the public copy.] The linear filter model of a synapse.

Write the input spike train as a sum of delta functions,

$$\rho_b(t) = \sum_i \delta(t - t_i)$$

where the t_i are the spike times. Then the synaptic conductance is that spike train filtered by K:

$$g_b(t) = g_{b,\max} \sum_{t_i<t} K(t-t_i) = g_{b,\max} \int_{-\infty}^{t} K(t-\tau)\,\rho_b(\tau)\,d\tau$$

The idea in plain words

The integral looks worse than it is. In plain terms: **every time a spike arrives, stamp down a copy of K starting at that moment, and add up all the copies**. Where spikes come close together the copies overlap and the conductance builds up; where they are sparse it decays back toward zero.
The delta functions are only a device for writing "stamp a copy here" as an integral. Nothing deep is hiding in them.

Watch out — this is the Week 2 linear filter, returning

Convolving an input with a kernel is exactly what the temporal filtering of Week 2 did. There the input was a stimulus and the kernel a receptive field; here the input is a spike train and the kernel is the synaptic time course. Same mathematics, different objects.

Watch out — linear in the spike train is not linear in the voltage

It is worth being precise about *what* is linear here. The conductance g_b is a linear function of the *input spike train* — stamp more spikes, get proportionally more conductance.
The synapse's effect on the membrane is not linear in V, because the synaptic current $g_s(V - E_s)$ multiplies the conductance by the voltage. That product is exactly where the nonlinearity of a real synapse lives.

7. A Two-Neuron Network

Everything is now in place. Take two integrate-and-fire neurons, connect each to the other, and model the synapses with alpha functions.

[Lecture slide omitted from the public copy.] Two neurons wired to each other. Left: excitatory synapses ($E_s = 0\,mV$) give alternating *firing. Right: inhibitory synapses ($E_s = -80\,mV$) give* synchrony.

Each neuron obeys the equation built above,

$$\tau_m \frac{dV}{dt} = -\Big[(V - E_L) + g_s(t)\,(V - E_s)\,r_m\Big] + I_e R_m$$

with $E_L = -70\,\text{mV}$, $V_{\text{thresh}} = -54\,\text{mV}$, $\tau_m = 20\,\text{ms}$, $\tau_{\text{peak}} = 10\,\text{ms}$ and $I_e R_m = 25\,\text{mV}$.

Watch out — the surprising result is the inhibitory one

Excitation producing alternation is intuitive enough — one cell fires, pushes the other over threshold, and they take turns.
Inhibition producing synchrony is not obvious at all. Mutual inhibition might be expected to force the cells apart, yet they end up firing together. The mechanism is that inhibition suppresses both cells at once and then wears off at the same rate, so they are released from it simultaneously and recover in lockstep.

The idea in plain words

This is the payoff for the whole lecture. Two neurons, one equation each, one parameter changed — E_s from 0 to $-80\,\text{mV}$ — and the network's collective behaviour changes qualitatively.

Synchrony arising from inhibition is not a curiosity either: it is a candidate mechanism for the rhythms seen in real cortical circuits.

8. Where This Leaves Us

- A synapse enters the membrane equation as one more conductance branch, $g_s(V - E_s)$ — the same form as every ionic current in Week 5.
- **Excitatory** versus **inhibitory** is decided by a single parameter: whether E_s lies above or below E_L.
- The conductance factors as $g_s = g_{s,\max} P_{\text{rel}} P_s$, and P_s follows the same two-state kinetic equation as Hodgkin–Huxley gating — driven by transmitter rather than voltage.
- Single-spike time courses are exponential for AMPA and GABA_A, alpha-shaped for NMDA, with timescales differing by an order of magnitude.
- Multiple spikes are handled by **linear filtering**: stamp a copy of the kernel at each spike time and sum.
- Wiring two integrate-and-fire neurons together gives *alternating* firing when the synapses are excitatory and *synchrony* when they are inhibitory.

> *To add from lectures: Next: moving from spiking networks to **firing-rate** models, which make it far cheaper to simulate large networks of neurons. A supplementary handout on recurrent networks accompanies this week.*

Part II

From Spikes to Firing Rates

Lecture 2 — trading spike timing for tractability, and what a feedforward network computes.

9. Spiking or Firing Rate?

*[Lecture slide omitted from the public copy.] The two network architectures this week builds toward: **feedforward** (input → output through W) and **recurrent** (outputs also connected to each other through M).*

Before building either, a modelling choice has to be made.

[Lecture slide omitted from the public copy.] The trade-off, stated plainly.

	Advantages	**Disadvantages**
Spiking	spike timing, synchrony, correlations	computationally expensive
Firing rate	efficient, scales to large networks	ignores spike timing

10. Bridging the Two

The bridge is the linear filter from lecture 1.

[Lecture slide omitted from the public copy.] Recap: a spike train convolved with a synaptic kernel gives a conductance.

[Lecture slide omitted from the public copy.] With N synapses, and assuming no nonlinear interaction between them, the total synaptic current is just the weighted sum.

$$I_s(t) = \sum_{b=1}^{N} w_b \int_{-\infty}^{t} K(t-\tau)\,\rho_b(\tau)\,d\tau$$

The leap

[Lecture slide omitted from the public copy.] Replace each spike train $\rho_b(t)$ with the input neuron's instantaneous firing rate $u_b(t)$.

Watch out — this replacement is an approximation, and it can fail

Swapping a spike train for a smooth rate discards the actual spike times. It works when the inputs are effectively independent — but breaks down precisely when there are **correlations** or **synchrony** between input neurons, since a rate cannot express "these two fired together". Which is the same limitation the table above named. The synchrony result at the end of lecture 1 is exactly the kind of phenomenon this model cannot see.

Getting rid of the integral

Choose the synaptic kernel to be exponential, $K(t) = \frac{1}{\tau_s} e^{-t/\tau_s}$, and differentiate both sides. The integral disappears:

[Lecture slide omitted from the public copy.] Substituting an exponential filter and differentiating turns the convolution into a differential equation.

$$\tau_s \frac{dI_s}{dt} = -I_s + \mathbf{w} \cdot \mathbf{u}$$

The idea in plain words

Note the shape: it is the RC membrane equation again, with I_s in place of V and the weighted input $\mathbf{w} \cdot \mathbf{u}$ in place of $I_e R_m$. The synaptic current relaxes toward the weighted sum of inputs with time constant τ_s.
Third time this form has appeared — membrane, gating, and now synaptic current.

11. The Firing-Rate Model

[Lecture slide omitted from the public copy.] The full model: one equation for the output rate, one for the synaptic current.

$$\tau_r \frac{dv}{dt} = -v + F\big(I_s(t)\big), \qquad \tau_s \frac{dI_s}{dt} = -I_s + \mathbf{w} \cdot \mathbf{u}$$

where F is an optional nonlinearity.

Two regimes

- **Fast synapses** ($\tau_s \ll \tau_r$): I_s settles almost instantly, so $I_s = \mathbf{w}\cdot\mathbf{u}$ and $\tau_r\, dv/dt = -v + F(\mathbf{w}\cdot\mathbf{u})$.
- **Fast output** ($\tau_r \ll \tau_s$): v tracks the current, so $v = F(I_s(t))$ with I_s carrying the dynamics.

Watch out — you have seen this equation before, in another field

$v_{ss} = F(\mathbf{w}\cdot\mathbf{u})$ — a weighted sum passed through a nonlinearity — is precisely the artificial neuron of machine learning, with F a sigmoid or threshold.
The point is what it *omits*. It is the steady state of a system with real dynamics on both the synaptic current and the output rate, which are themselves an approximation to spiking neurons. The neural-network unit is the endpoint of a long chain of simplifications, each discarding something.

12. Many Outputs, and Feedback

[Lecture slide omitted from the public copy.] Scaling up: the output becomes a vector $\mathbf{v}$ *and the weight vector becomes a matrix* W*.*

$$\tau \frac{d\mathbf{v}}{dt} = -\mathbf{v} + F(W\mathbf{u})$$

Index order in W

W_{ij} is the synaptic weight **from** input neuron j **to** output neuron i — destination first, source second.

[Lecture slide omitted from the public copy.] Adding connections among *the output neurons, with strengths in a second matrix* M*.*

$$\tau \frac{d\mathbf{v}}{dt} = \underbrace{-\mathbf{v}}_{\text{decay}} + F\big(\underbrace{W\mathbf{u}}_{\text{input}} + \underbrace{M\mathbf{v}}_{\text{feedback}}\big)$$

The idea in plain words

Setting $M = 0$ recovers the feedforward network exactly, so feedforward is the special case, not a separate model.
The lecture's aside is worth keeping: a purely feedforward brain would react to every stimulus immediately and say things it should not. Recurrence is what lets a network hold state and modulate its own response.

13. What a Linear Feedforward Network Computes

Drop the nonlinearity and hold the input static. Setting $d\mathbf{v}/dt = 0$ gives simply

$$\mathbf{v}_{ss} = W\mathbf{u}$$

[Lecture slide omitted from the public copy.] A worked example: a 6×5 *weight matrix and a* 5×1 *input.*

[Lecture slide omitted from the public copy.] The output. What is this network doing?

[Lecture slide omitted from the public copy.] The same thing plotted: a trapezoidal input becomes a pair of opposite-signed spikes at its edges.

[Lecture slide omitted from the public copy.] The same operation on a 2D image, by flattening it row by row into one long vector $\mathbf{u}$*.*

14. The Brain Does Calculus

[Lecture slide omitted from the public copy.] Receptive fields in primary visual cortex — the edge detectors from Week 1.

[Lecture slide omitted from the public copy.] Why those receptive field shapes are derivative operators.

Write down the definition of a derivative and discretise it:

$$\frac{df}{dx} \approx f(x+1) - f(x) \qquad \longleftrightarrow \qquad \text{filter } [-1 \quad 1]$$

which is exactly the edge-detecting filter above. Do the same for the second derivative:

$$\frac{d^2 f}{dx^2} \approx f(x+1) - 2f(x) + f(x-1) \qquad \longleftrightarrow \qquad \text{filter } [1 \quad -2 \quad 1]$$

and the coefficients $1, -2, 1$ are precisely the centre–surround "bar detector" receptive field.

The idea in plain words

So V1's two classic receptive field types are the first and second spatial derivatives of the image — the edge detector differentiates once, the bar detector twice. And since images are two-dimensional, oriented receptive fields compute those derivatives along different directions.
The brain was doing calculus long before anyone wrote it down.

15. Where This Leaves Us

- Choosing **spiking** or **firing-rate** neurons trades the ability to model timing and synchrony against the ability to simulate large networks.
- The bridge is to replace each spike train ρ_b with a firing rate u_b — valid when inputs are uncorrelated, invalid exactly when synchrony matters.
- With an exponential synaptic kernel the convolution becomes $\tau_s \, dI_s/dt = -I_s + \mathbf{w} \cdot \mathbf{u}$ — the RC equation once more.
- Setting *all* time derivatives to zero gives $v_{ss} = F(\mathbf{w} \cdot \mathbf{u})$, the artificial neuron of machine learning, revealed as the steady state of a much richer system.
- Networks are $\tau \, d\mathbf{v}/dt = -\mathbf{v} + F(W\mathbf{u} + M\mathbf{v})$, with **feedforward** the special case $M = 0$. Negative off-diagonal entries in M implement mutual inhibition.

- A linear feedforward network whose weight matrix holds shifted copies of $[-1, 1]$ computes a **spatial derivative** — which is what V1 simple cells appear to do, with the $[1, -2, 1]$ receptive field computing the second derivative.

To add from lectures: Next: recurrent networks proper, analysed with eigenvectors and eigenvalues. The supplementary handout in this folder works through the differential equation for a linear recurrent network.

Part III

Recurrent Networks

Lecture 3 — eigenvalues and stability, amplification, memory, and what happens when the network is neither linear nor symmetric.

16. The Question

[Lecture slide omitted from the public copy.] The equation to solve, writing $\mathbf{h} = W\mathbf{u}$ for the feedforward input.

$$\tau \frac{d\mathbf{v}}{dt} = -\mathbf{v} + \mathbf{h} + M\mathbf{v}$$

The idea in plain words

The difficulty is that $\mathbf{v}$ appears on both sides, coupled through a matrix. Each neuron's rate of change depends on every other neuron's current rate, so the equations cannot be solved one at a time.
The whole lecture is about one trick for untangling that.

17. Eigenvectors to the Rescue

[Lecture slide omitted from the public copy.] The plan: use the eigenvectors of M to decouple the equation.

Assume M is symmetric

$M_{ij} = M_{ji}$ — if neuron 1 connects to neuron 2 with strength a, then 2 connects back to 1 with the same a.
A symmetric $N \times N$ matrix has N **orthogonal** eigenvectors satisfying $M\mathbf{e}_i = \lambda_i \mathbf{e}_i$, and they can be scaled to be **orthonormal**: $\mathbf{e}_i \cdot \mathbf{e}_j = 0$ for $i \neq j$, and $\mathbf{e}_i \cdot \mathbf{e}_i = 1$.

[Lecture slide omitted from the public copy.] Rewriting the output vector in the eigenvector basis, and the closed-form solution that results.

Because the eigenvectors are orthonormal they form a *coordinate system*, so any output vector can be written in terms of them:

$$\mathbf{v}(t) = \sum_{i=1}^{N} c_i(t)\, \mathbf{e}_i$$

Substituting this into the differential equation and using $M\mathbf{e}_i = \lambda_i \mathbf{e}_i$ gives a separate, ordinary equation for each coefficient, with solution

$$c_i(t) = \frac{\mathbf{h} \cdot \mathbf{e}_i}{1 - \lambda_i} \left(1 - e^{-t(1-\lambda_i)/\tau}\right) + c_i(0)\, e^{-t(1-\lambda_i)/\tau}$$

The idea in plain words

This is the payoff for the change of basis. In the original coordinates the N neurons are all tangled together; in the eigenvector basis each coefficient c_i evolves *independently*, and each one obeys the same familiar exponential relaxation we have seen since Week 5.
The only thing distinguishing one mode from another is its eigenvalue λ_i — which appears in both the size of the response and the speed of the approach.

18. Eigenvalues Determine Stability

[Lecture slide omitted from the public copy.] Reading stability off the eigenvalues.

Look at the exponent $-t(1 - \lambda_i)/\tau$:

- If any $\lambda_i > 1$, then $1 - \lambda_i < 0$ and the exponent is *positive*. That term grows without bound — the network is **unstable** and $\mathbf{v}(t)$ explodes.
- If all $\lambda_i < 1$, every exponential decays and the network settles at

$$\mathbf{v}_{ss} = \sum_i \frac{\mathbf{h} \cdot \mathbf{e}_i}{1 - \lambda_i} \mathbf{e}_i$$

Watch out — $\lambda = 1$ is the knife edge, and it is not a degenerate case

At $\lambda_i = 1$ the denominator $1 - \lambda_i$ vanishes and the formula above breaks down. That is not a failure of the model — it is a genuinely different regime, and it turns out to be the useful one. Section 22 returns to it.

19. Amplification

[Lecture slide omitted from the public copy.] When one eigenvalue is close to 1, its term dominates the sum.

Suppose all $\lambda_i < 1$ but λ_1 is close to 1 while the rest are much smaller. Then the $i = 1$ term swamps the others:

$$\mathbf{v}_{ss} \approx \frac{\mathbf{h} \cdot \mathbf{e}_1}{1 - \lambda_1} \mathbf{e}_1$$

The idea in plain words

The factor $1/(1 - \lambda_1)$ is an **amplification gain**. With $\lambda_1 = 0.9$ it is $1/0.1 = 10$, so the network multiplies the projection of its input onto $\mathbf{e}_1$ tenfold.
And it is selective: only the component of the input lying along $\mathbf{e}_1$ gets amplified. Everything else is divided by a number close to 1 and passes through roughly unchanged. The network

is picking out one pattern and boosting it.

20. A Concrete Example: the Cosine Network

[Lecture slide omitted from the public copy.] Label each output neuron with an angle from $-180°$ to $+180°$, and set the recurrent weights by the relative *angle: $M(\theta, \theta') \propto \cos(\theta - \theta')$.*

The idea in plain words

Cosine connectivity means **nearby neurons excite each other and distant ones inhibit** — positive weight for small angular differences, negative for large. And it is automatically symmetric, since $\cos(x) = \cos(-x)$, so all the eigenvector machinery applies.
This is not an arbitrary choice: there is evidence for exactly this pattern in cerebral cortex, where neurons tend to excite their neighbours and inhibit cells further away.

[Lecture slide omitted from the public copy.] Simulation with all eigenvalues zero except $\lambda_1 = 0.9$. A noisy input becomes a cleanly amplified peak, with the smaller bumps suppressed — the predicted factor of 10.

21. Memory: What Happens at $\lambda = 1$

[Lecture slide omitted from the public copy.] Setting $\lambda_1 = 1$ exactly, with all other eigenvalues below 1.

With $\lambda_1 = 1$ the equation for that coefficient loses its decay term entirely:

$$\tau \frac{dc_1}{dt} = \mathbf{h} \cdot \mathbf{e}_1 \qquad \Longrightarrow \qquad \mathbf{v}(t) \approx c_1 \mathbf{e}_1 = \frac{\mathbf{e}_1}{\tau} \int_0^t \mathbf{h}(t') \cdot \mathbf{e}_1 \, dt'$$

The idea in plain words

There is no $-v$ pulling the mode back to zero any more, so c_1 simply *accumulates* its input. Turn the input on and then off, and the network keeps whatever value it had reached — **sustained activity with no input at all**.
The network has become an **integrator**, holding a memory of the integral of everything it has received.

[Lecture slide omitted from the public copy.] Integrator neurons in the medial vestibular nucleus. Bursts from oculomotor neurons step the eye position up and down; the integrator neuron holds the resulting position as persistent firing.

The idea in plain words

Week 6 lecture 2 showed the brain differentiating — V1 receptive fields as first and second spatial derivatives. This is the other half: a recurrent circuit performing *integration*, holding eye position between saccades.
Differentiation in the feedforward weights, integration in the recurrent ones.

22. Nonlinear Recurrent Networks

[Lecture slide omitted from the public copy.] Adding a nonlinearity to the recurrent sum: $\tau\, d\mathbf{v}/dt = -\mathbf{v} + F(\mathbf{h} + M\mathbf{v})$.

Rectification

$F(x) = [x]_+ = x$ if $x > 0$, and 0 otherwise.
This is not merely convenient. $\mathbf{v}$ represents *firing rates*, which cannot be negative, so rectification is what keeps the model physically meaningful.

[Lecture slide omitted from the public copy.] The nonlinear network amplifying and cleaning up a noisy input — with $\lambda_1 = 1.9$.

Watch out — rectification rescues an eigenvalue that would otherwise explode

$\lambda_1 = 1.9$ is greater than 1. In the *linear* network that guarantees instability and unbounded growth.
Here the network is perfectly stable, because the rectification clips the runaway before it can compound. A nonlinearity introduced for biological realism turns out to change the stability analysis fundamentally — the linear criterion no longer applies.

Selective attention

[Lecture slide omitted from the public copy.] Two peaks go in; one comes out. The network performs ***winner-take-all*** *selection.*

Gain modulation

[Lecture slide omitted from the public copy.] Adding a constant *to the input* multiplies *the output.*

The idea in plain words

Note how odd that is. The inputs differ by an additive offset — three curves of identical shape, shifted vertically. The outputs differ by a multiplicative factor — same shape, scaled. An additive change becoming a multiplicative one is exactly **gain modulation**, and it has been observed in area 7a of parietal cortex.

Memory, again

[Lecture slide omitted from the public copy.] Top: local input plus background, giving an amplified peak. Bottom: the local input switched off — and the peak persists, at the same location.

The idea in plain words

The network sustains its own activity through recurrent excitation once the input is gone, and it remembers *where* the bump was. That is a content-addressable short-term memory, and it is a standard model for the persistent activity seen in prefrontal cortex during working-memory tasks.

23. Non-Symmetric Networks

Everything so far assumed M symmetric. The simplest case where that fails is also the most biologically natural.

[Lecture slide omitted from the public copy.] An excitatory neuron and an inhibitory neuron, each connected to the other.

Watch out — excitatory–inhibitory pairs cannot be symmetric

If E excites I, that weight is positive. If I inhibits E, that weight is negative. Since $M_{EI} \neq M_{IE}$ by sign alone, the matrix cannot be symmetric — and the orthogonal-eigenvector machinery no longer applies.

The two coupled equations, with rectification and thresholds:

$$\tau_E \frac{dv_E}{dt} = -v_E + \left[M_{EE}v_E + M_{EI}v_I - \gamma_E\right]_+$$
$$\tau_I \frac{dv_I}{dt} = -v_I + \left[M_{II}v_I + M_{IE}v_E - \gamma_I\right]_+$$

with $M_{EE} = 1.25$, $M_{EI} = -1$, $M_{IE} = 1$, $M_{II} = 0$, $\gamma_E = -10$, $\gamma_I = 10$, $\tau_E = 10\,\text{ms}$ — and τ_I left free.

24. Linear Stability Analysis

*[Lecture slide omitted from the public copy.] Differentiating the right-hand sides with respect to v_E and v_I gives the **stability matrix**, also called the Jacobian.*

The idea in plain words

The idea is to ask what happens *near* a fixed point — the (v_E, v_I) where both derivatives vanish. Close enough to it, the nonlinear system behaves like a linear one, and that linear approximation is the Jacobian.
Because the Jacobian is *not* symmetric, its eigenvalues can be complex. The **real part** decides stability; the **imaginary part** produces oscillation.

[Lecture slide omitted from the public copy.] $\tau_I = 30\,ms$ makes the real parts negative. Both rates oscillate but the oscillations are damped, and the trajectory spirals into the fixed point.

Point attractor

A stable fixed point that nearby trajectories converge onto. Complex eigenvalues with negative real part give the spiralling, damped approach seen here.

[Lecture slide omitted from the public copy.] $\tau_I = 50\,ms$ makes the real parts positive. Trajectories now spiral away from the fixed point — but they do not escape.

The idea in plain words

Instability would normally mean unbounded growth. It does not happen here for the same reason as before: as v_E is driven negative, **rectification clips it at zero** and pushes it

back. Growth outward, clipping inward, and the trajectory settles onto a closed loop — a **limit cycle**.
The result is a stable, self-sustaining oscillation. Not a decaying one, and not a runaway.

Hopf bifurcation

Changing one parameter — here τ_I from 30 to 50 ms — takes the network from a stable fixed point to a limit cycle. A qualitative change in behaviour produced by a smooth change in a parameter is called a **bifurcation**; this particular one, fixed point to oscillation, is a Hopf bifurcation.

25. Where This Leaves Us

- A symmetric M has orthonormal eigenvectors, and writing $\mathbf{v} = \sum_i c_i \mathbf{e}_i$ **decouples** the network into independent modes, each relaxing exponentially.
- **Eigenvalues set stability**: any $\lambda_i > 1$ and the linear network explodes; all below 1 and it converges.
- λ_1 near 1 gives **amplification** by $1/(1-\lambda_1)$, selectively along $\mathbf{e}_1$. At exactly $\lambda_1 = 1$ the mode becomes an **integrator** and the network holds a memory of past input — as real integrator neurons do for eye position.
- **Rectification** changes everything: a nonlinear network stays stable with $\lambda_1 = 1.9$, and additionally performs winner-take-all selection, gain modulation, and working-memory-like persistence.
- Excitatory–inhibitory circuits are inherently **non-symmetric**. Near a fixed point they are analysed through the **Jacobian**, whose complex eigenvalues have a real part governing stability and an imaginary part governing oscillation.
- Varying one time constant carries the network from a **point attractor** through a **Hopf bifurcation** to a **limit cycle** — with rectification again supplying the bound that makes the oscillation stable rather than explosive.

To add from lectures: Next week: synaptic plasticity and learning — how the brain changes the connections W and M that this week took as given.